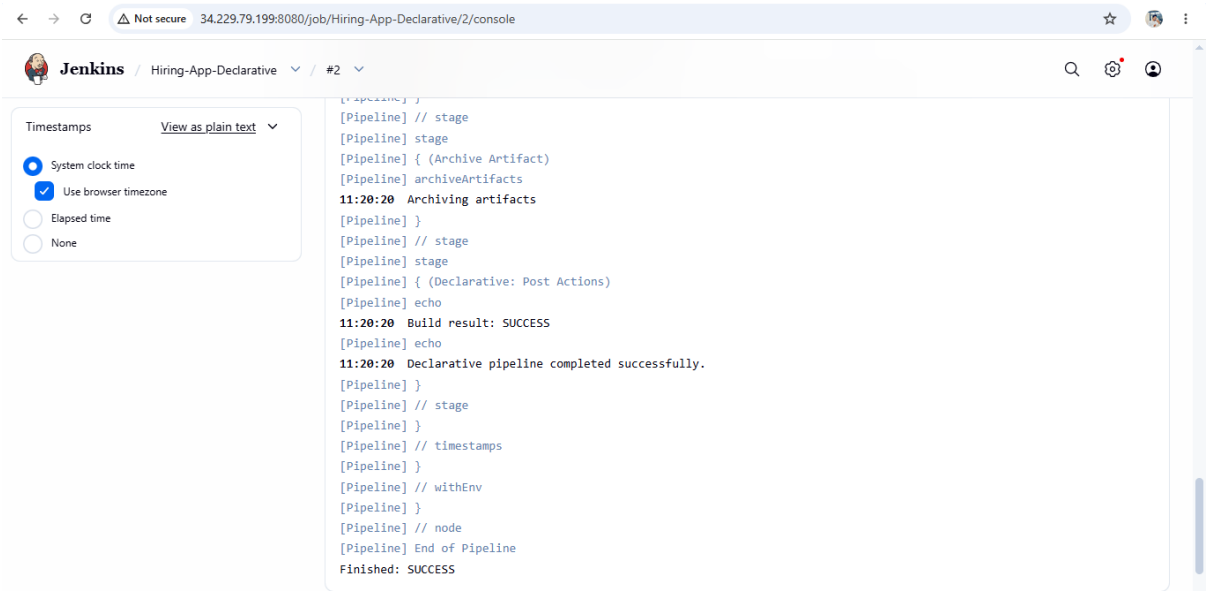
Jenkins CI/CD Pipeline Implementation Using Declarative, Scripted, Multi-Stage, and Parallel Pipelines

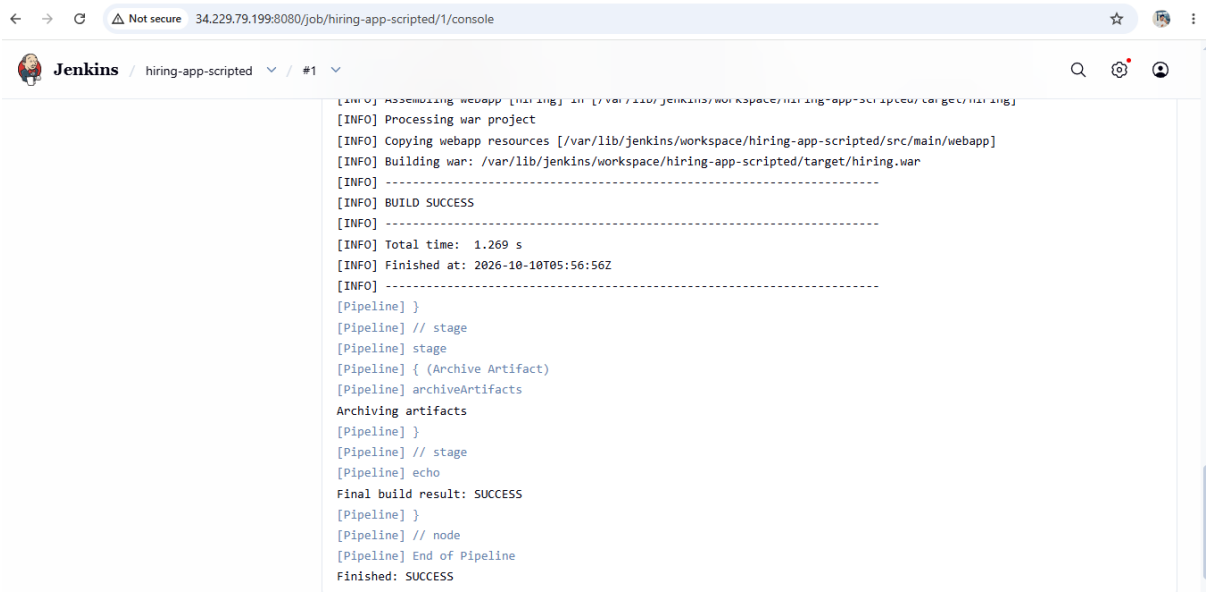
Job 1 — Declarative Pipeline

Declarative pipelines use a structured `pipeline {}` block and are generally easier to read and maintain.



Job 2 — Scripted Pipeline

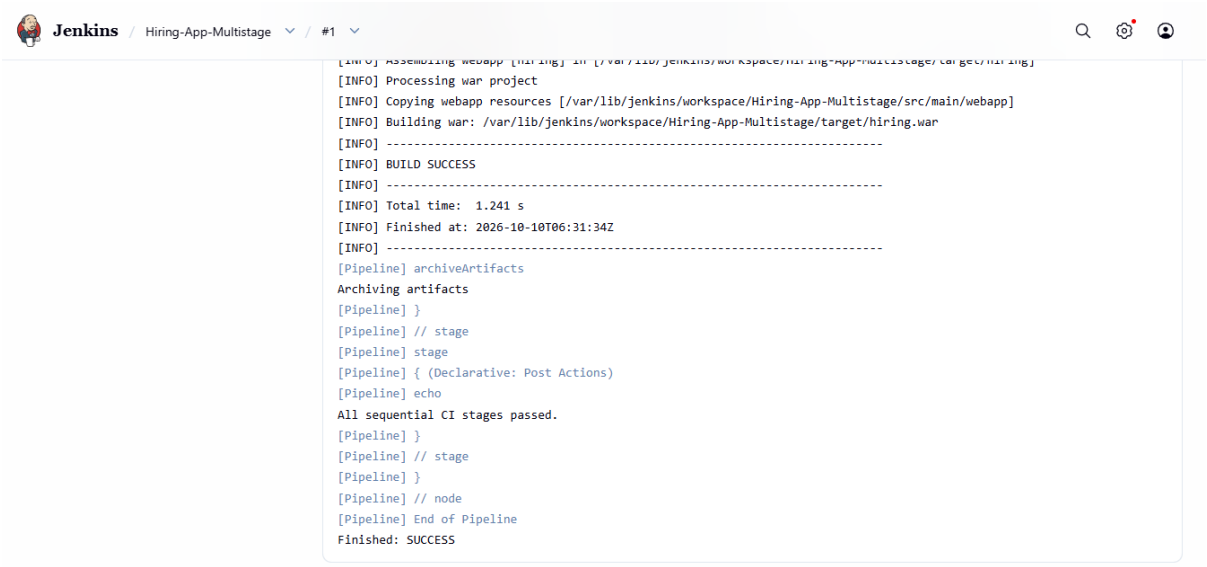
Scripted pipelines use Groovy code inside a `node {}` block. This version uses `try/catch/finally` to handle build failures.



Difference: Scripted Pipeline gives you more direct Groovy control over conditions, loops, error handling, and dynamic execution. Declarative Pipeline provides a more structured syntax.

Job 3 — Multi-stage Pipeline

A multi-stage pipeline organizes the CI/CD lifecycle into separate stages that run sequentially. This job adds testing and packaging to the workflow.



Job 4 — Parallel Stage Pipeline

Parallel execution lets independent tasks run concurrently, potentially reducing the overall build time. This example checks the source and Java/Maven environment in parallel after checkout, then packages the application.

